

PROJECT FILE

Auto Ticket Classification using Flow

Executable reference • Python + scikit-learn • Text classification

PROJECT EXECUTABLE FILES

Prepared as an academic project reference

1. Project Overview

This reference implementation classifies incoming support tickets into categories such as Billing, Technical Support, Account Access, and Delivery. It uses a TF-IDF text representation and a Logistic Regression classifier. The included script trains from a CSV file and predicts a category for a new ticket.

2. Folder Structure

```
AutoTicketClassification/
■■■ tickets.csv           # training examples
■■■ train_and_predict.py  # executable Python script
■■■ ticket_model.joblib   # generated after training
```

3. Install Requirements

```
python -m pip install pandas scikit-learn joblib
```

4. Training Dataset (tickets.csv)

Create a CSV with two columns: **text** and **category**. Example rows:

```
text,category
"I was charged twice for my subscription",Billing
"The app crashes when I open settings",Technical Support
"I cannot reset my password",Account Access
"My package has not arrived",Delivery
"Please explain the invoice amount",Billing
"The screen freezes after login",Technical Support
"My account is locked",Account Access
"Track my delayed order",Delivery
```

5. Executable Python File

Save the following as **train_and_predict.py** in the same folder as tickets.csv.

```
import pandas as pd
import joblib
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

DATA_FILE = Path("tickets.csv")
MODEL_FILE = Path("ticket_model.joblib")

def train_model():
    data = pd.read_csv(DATA_FILE)
    data = data.dropna(subset=["text", "category"])
    data["text"] = data["text"].astype(str)
    data["category"] = data["category"].astype(str)

    if data["category"].nunique() < 2:
        raise ValueError("Training data must contain at least 2 categories.")

    model = Pipeline([
        ("tfidf", TfidfVectorizer(ngram_range=(1, 2), stop_words="english")),
        ("classifier", LogisticRegression(max_iter=1000))
    ])

    # Use a stratified split only when every category has at least 2 samples.
    counts = data["category"].value_counts()
    if len(data) >= 4 and counts.min() >= 2:
        x_train, x_test, y_train, y_test = train_test_split(
            data["text"], data["category"], test_size=0.25,
            random_state=42, stratify=data["category"]
        )
        model.fit(x_train, y_train)
        print(classification_report(y_test, model.predict(x_test), zero_division=0))
    else:
```

```

model.fit(data["text"], data["category"])
print("Small dataset: trained on all rows; add more examples to evaluate.")

joblib.dump(model, MODEL_FILE)
print(f"Model saved to: {MODEL_FILE}")

def predict_ticket(ticket_text):
    model = joblib.load(MODEL_FILE)
    predicted = model.predict([ticket_text])[0]
    probabilities = model.predict_proba([ticket_text])[0]
    confidence = float(max(probabilities))
    print(f"Predicted category: {predicted}")
    print(f"Model confidence: {confidence:.2%}")
    return predicted, confidence

if __name__ == "__main__":
    if not MODEL_FILE.exists():
        train_model()

    ticket = input("Enter a support ticket: ").strip()
    if ticket:
        predict_ticket(ticket)
    else:
        print("No ticket text entered.")

```

6. Run the Project

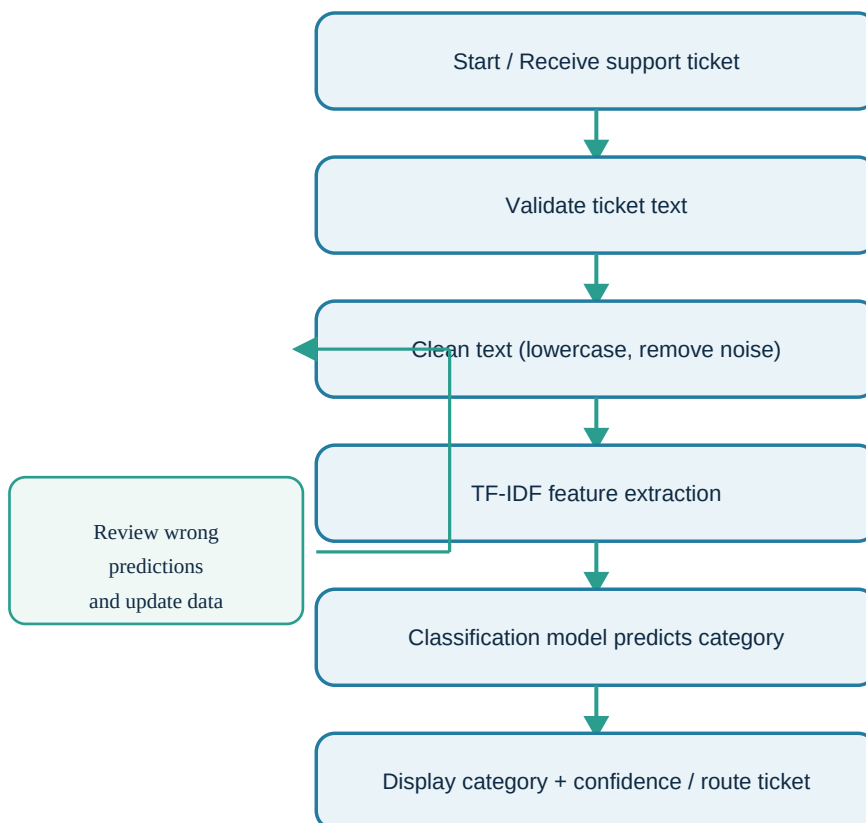
```

python train_and_predict.py
Enter a support ticket: I cannot access my account
Predicted category: Account Access
Model confidence: (varies with training data)

```

Note: The sample dataset is intentionally small for demonstration. Add many real, correctly labelled examples per category before using this model in a live help desk. Confidence is a model score, not a guarantee of correctness.

7. Main Processing Flow



Ticket input → text validation and cleaning → TF-IDF feature extraction → trained classifier → predicted category
→ support-team routing and feedback.